

Introducing Rust

**The Safe and Fast
Future of Programming**

I promise it's not a cult!

By Alex Bissessur



alex.yaml

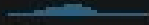
```
---
apiVersion: v1
kind: Person
metadata:
  name: Alex Bissessur
spec:
  work:
    company: La Sentinelle
    role: Kubernetes Person
    location: Mauritius
  contact:
    website: alexbissessur.dev
    mastodon: moris.social/@AlexB
    github: github.com/xelab04
  interests:
    - Kubernetes
    - Linux
    - Free & Open Source Software
  hobbies:
    - Playing kubectl with Homelab
```

“I do fun things with
Kubernetes.”



Let Me Complain

- Modern software is bloated and inefficient
- Programming languages treat optimisation as an afterthought
- Code is written to work first, with performance a “future thing”

Name	CPU	Memory
 Vesktop		663.4 MiB
 Chromium Web Browser		337.8 MiB
org.chromium.Chromium		233.4 MiB

Let Me Complain

- Electron is the default for desktop apps, based on Chromium
- Duck typing and garbage collection, while not noticeable, contribute to slow code
- Memory inefficiencies force expensive scalable cloud infra
- Devs have to choose between dev velocity (Python, JS, PHP) over performance (C, C++)

What is Rust?

- A programming language built to fit a niche
 - C and C++: fast and performant but prone to bugs
 - Java/Js/Python: slow and resource hungry w/ less bugs
- Started as a personal project born from frustration at a broken elevator
- Sponsored by Mozilla (2009) and the Rust Foundation (2021)
- Appears everywhere – AWS, Linux, Ladybird, Clownflare

Some Facts About Rust

- No garbage collector, unlike Go, Java, NodeJS
- Memory safety through ownership
- Compiles to a binary as performant as C
- World's best compiler w/ detailed errors & suggestions
- Has a package manager, tests, formatting, and docs, just like any modern language

Let's Write Some Rust

```
1 fn main() {  
  println!("Hello World!");  
2 }  
3
```

```
alex@workpad ~/P/S/D/2/demo (main)> rustc main.rs  
alex@workpad ~/P/S/D/2/demo (main)> ls  
main*  main.rs  
alex@workpad ~/P/S/D/2/demo (main)> ./main  
Hello World!  
alex@workpad ~/P/S/D/2/demo (main)>
```

Rust with Cargo

- rustc drawbacks: managing dependencies, linking libraries, build flags etc etc
- Cargo is Rust's version of NPM or Pip
 - package manager
 - build system
 - runs tests
 - generates docs

Rust with Cargo

```
alex@workpad ~/P/S/D/2/demo (main)> cargo new demo
    Creating binary (application) `demo` package
note: see more `Cargo.toml` keys and their definitions at
cargo/reference/manifest.html
alex@workpad ~/P/S/D/2/demo (main)> ls demo/
Cargo.toml  src/
alex@workpad ~/P/S/D/2/demo (main)> ls demo/src/
main.rs
alex@workpad ~/P/S/D/2/demo (main)>
```

Rust with Cargo

```
alex@workpad ~/P/S/D/2/d/demo (main)> cargo add log
Updating crates.io index
Adding log v0.4.33 to dependencies
Features:
- kv
- kv_serde
Updating crates.io index
Locking 1 package to latest Rust 1.95.0 compatible version
```

```
alex@workpad ~/P/S/D/2/d/demo (main)> cat Cargo.toml
[package]
name = "demo"
version = "0.1.0"
edition = "2024"

[dependencies]
log = "0.4.33"
```

Rust Syntax

- Variables & Immutability
- Functions
- Structs & Enums
- Pattern Matching
- Using Libraries

Ownership

- The Problem:
 - manual memory (C/C++) handling prone to leaks, double frees, use-after-free
 - garbage collection is safe but introduces runtime overhead and unpredictable latency
- Rust's Ownership
 - each value has an owner
 - only one owner at a time
 - when owner goes out of scope, memory is automatically freed

Borrowing

```
alex@workpad ~/P/S/D/2/d/demo (main) [101]> cat src/main.rs
// this function borrows the value and
// returns ownership to the parent function afterwards
fn borrow_print(text: &String) {
    println!("borrowed {text}");
}

// the ownership is passed into this function
fn own_print(text: String) {
    println!("owned {text}");
}

fn main() {
    let s = String::from("hello");

    borrow_print(&s);

    own_print(s);

    borrow_print(&s);
}
```

Web Development in Rust

- The components: Actix-web, Axum, Rocket
- Micro Frameworks
 - Similar to Flask
 - Fast & flexible, but needs a LOT of assembly for: ORM, templating, auth, sessions, db migrations
 - Complex setup, painful boilerplate
- What we need:
 - A batteries-included framework
 - Similar to Django, Laravel, or Rails

Cot

- Rust types and structs as source of truth with query validation
- DB engine agnostic, auto-generated migrations
- Built-in Admin Panel for all CRUD operations
- Automatic form validation with type safety
- Built-in authentication with sessions, auth, and security middleware
- Automated OpenAPI/Swagger docs generation

Demo: Rewrite MeetupMU in Rust

Learning Rust

Rustlings

(<https://rustlings.rust-lang.org/>)



The Rust Book

(<https://doc.rust-lang.org/stable/book/>)

